

The `-guess` Option in CBC/CLP: Automatic LP Parameter Selection

Auto-generated documentation from source code analysis

March 2026

Abstract

The `-guess` command in CBC (and CLP) inspects structural properties of the loaded LP model and returns a string of solver parameters predicted to yield good performance. This document describes the decision logic implemented in `ClpSimplexOther::guess()`, the features it examines, and the parameter configurations it produces. A decision tree diagram summarises the complete logic.

Contents

1 Overview	1
2 Features Examined	2
3 Decision Logic	2
4 Decision Tree Diagram	3
5 Parameter Descriptions	3
5.1 LP Algorithm Selection	3
5.2 Idiot Crash	3
5.3 Perturbation	3
5.4 Dual Pivot Strategy	3
6 Integration in CBC	4
7 Broader Context: OsiFeatures	4
8 Source Code Locations	4
9 Absence of Barrier and Future Extensions	4
9.1 Barrier Is Never Selected	4
9.2 When Barrier Would Be Appropriate	5
9.3 Candidate MIPLIB 2017 Instances for Barrier Experiments	5
9.4 Proposed Training Pipeline for an Improved Decision Tree	5

1 Overview

When the user issues the `guess` action inside the CBC interactive shell (or calls it through the C interface), the following chain of events occurs:

1. The current LP model is obtained from the `OsiClpSolverInterface`.

2. The model pointer is cast to `ClpSimplexOther`.
3. The method `ClpSimplexOther::guess(int mode)` is called.
4. The returned string of CLP command-line flags is injected into the command queue, overriding the default LP strategy for the subsequent solve.

The idea is rooted in the *algorithm selection* literature: different LP instances respond better to different solver configurations, and a small set of cheaply computable instance features can predict which configuration to use. The reference work underlying the feature infrastructure is:

Vilas Boas, M. G.; Santos, H. G.; Merschmann, L. H. C. and Vanden Berghe, G. *Optimal Decision Trees for the Algorithm Selection Problem: Integer Programming Based Approaches*. International Transactions in Operational Research, DOI 10.1111/itor.12724, 2019.

2 Features Examined

The `guess()` method computes three features from the model:

1. **allInt** — a Boolean flag that is **true** when every variable with a non-trivial bound range ($u_j > l_j$) is declared integer. Continuous variables whose lower and upper bounds coincide (fixed variables) are ignored.
2. **average** — the arithmetic mean of the objective function coefficients:

$$\text{average} = \frac{1}{n} \sum_{j=1}^n c_j$$

where n is the number of columns and c_j are the (sorted) objective coefficients.

3. **median** — the median objective coefficient, taken as the element at position $\lfloor n/2 \rfloor$ of the sorted objective vector.

These features are computed in $O(n \log n)$ time (dominated by the sort of the objective vector).

3 Decision Logic

The decision tree has depth two. The first split is on **allInt**; the second split depends on the branch:

- If **allInt** is true, the split is on $\text{average} \leq 0.0086207$.
- If **allInt** is false, the split is on $\text{median} \leq 0.75$.

This yields four leaf configurations, described in Table 1.

4 Decision Tree Diagram

5 Parameter Descriptions

5.1 LP Algorithm Selection

-**primals** Solve the LP relaxation using the primal simplex method. Chosen in three of the four leaves (L1, L2, L4).

-**duals** Solve the LP relaxation using the dual simplex method. Chosen only in leaf L3.

Table 1: Parameter configurations at each leaf of the decision tree.

Leaf	Condition	Parameters
L1	$\text{allInt} \wedge \text{average} \leq 0.0086207$	-idiot 30 -pertvalue -1483 -primals
L2	$\text{allInt} \wedge \text{average} > 0.0086207$	-idiot 60 -primals
L3	$\neg \text{allInt} \wedge \text{median} \leq 0.75$	-dualpivot pesteepest -psi 1.0 -pertv 52 -duals
L4	$\neg \text{allInt} \wedge \text{median} > 0.75$	-idiot 80 -primals

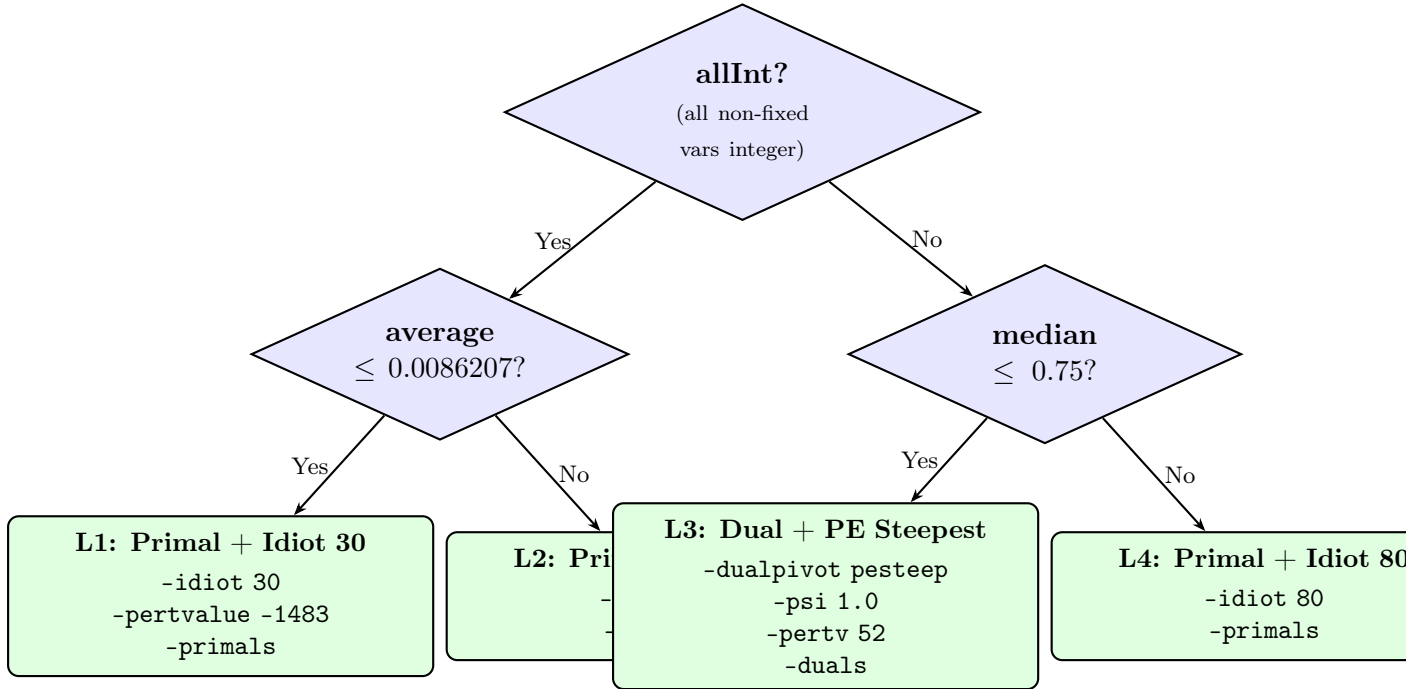


Figure 1: Decision tree implemented by `ClpSimplexOther::guess()`.

5.2 Idiot Crash

`-idiot  $k$` Run k iterations of the “idiot” crash procedure before starting simplex. The idiot crash is a Lagrangian-relaxation-based heuristic that produces a warm-start basis for the primal simplex. Higher values invest more effort in the crash phase. The tree selects 30, 60, or 80 iterations depending on the model class.

5.3 Perturbation

`-pertvalue  $v$` Sets the perturbation value used to break degeneracy. A negative value (here -1483) triggers a specific internal perturbation strategy in CLP. Used only in leaf L1.

`-pertv  $v$` Sets the perturbation value for the dual simplex (here 52). Used only in leaf L3.

5.4 Dual Pivot Strategy

`-dualpivot pesteepest` Use the Positive Edge steepest-edge dual pivot rule. This is a variant of the steepest-edge pivot that incorporates the positive edge criterion to improve dual simplex performance. Used only in leaf L3.

`-psi  $\psi$` The ψ parameter controls the weight given to the positive-edge criterion in the dual pivot selection. A value of 1.0 gives full weight to the positive-edge component. Used only in leaf L3.

6 Integration in CBC

The `guess` action is registered as a CLP parameter (`ClpParam::GUESS`). When triggered from the CBC shell, the code in `CbcSolver.cpp` performs:

```
ClpSimplexOther *model2 =  
    static_cast<ClpSimplexOther *>(lpSolver);  
std::string input = model2->guess(0);  
// inject into command queue
```

The returned string (e.g. `"-idiot 60 -primal"`) is parsed and pushed onto the input command queue, so the subsequent LP solve uses the recommended settings.

In the C interface (`Cbc_C_Interface.cpp`), the same logic is invoked when the LP method is set to `LPM_Auto`. The returned string is parsed to extract individual parameters (idiot passes, perturbation value, ψ , dual pivot strategy, and LP method) which are then stored in the model's parameter arrays.

7 Broader Context: OsiFeatures

While the current `guess()` implementation uses only three simple features (allInt, average objective, median objective), the CBC/Osi infrastructure includes a much richer feature extraction framework in `OsiFeatures` (207 features), covering:

- Problem dimensions (rows, columns, density, non-zeros).
- Variable types (binary, general integer, continuous, unbounded).
- Constraint classification (partitioning, packing, covering, cardinality, knapsack, bin-packing, flow, singleton, aggregation, precedence, variable bound, mixed-binary, general integer).
- Coefficient matrix statistics (min, max, mean, std. dev., ratio of largest to smallest absolute value, integrality).

- Objective function and RHS statistics.
- Row and column non-zero distribution histograms.

These features are used by the `CbcInstanceFeatures` class to write CSV files for offline algorithm selection studies. A natural extension would be to replace the hand-crafted decision tree in `guess()` with a learned model trained on these 207 features.

8 Source Code Locations

File	Role
<code>Clp/src/ClpSimplexOther.cpp</code> (line ~11037)	Implementation of <code>guess()</code>
<code>Cbc/src/CbcSolver.cpp</code> (line ~3613)	Invocation from CBC shell
<code>Cbc/src/Cbc_C_Interface.cpp</code> (line ~1879)	Invocation from C interface
<code>Osi/src/Osi/OsiFeatures.hpp</code>	Full feature enumeration (207 features)
<code>Osi/src/Osi/OsiFeatures.cpp</code>	Feature computation
<code>Cbc/src/CbcInstanceFeatures.cpp</code>	CSV export of features

9 Absence of Barrier and Future Extensions

9.1 Barrier Is Never Selected

The current `guess()` implementation never returns a barrier (interior point) configuration. All four leaves select either primal or dual simplex. The C interface parsing code in `Cbc_C_Interface.cpp` does check for `"-barrier"` in the returned string, but this branch is currently dead code—it was written in anticipation of a future extension.

9.2 When Barrier Would Be Appropriate

Barrier methods tend to outperform simplex on LP relaxations that exhibit one or more of the following characteristics:

- **Large and dense** constraint matrices (high non-zeros per row/column).
- **Highly degenerate** problems where simplex stalls due to many variables at bounds.
- **Network or flow structure** that yields favourable Cholesky factorisation sparsity.
- **High proportion of equalities**, which produce better-conditioned normal equations for the barrier method while causing degeneracy issues for dual simplex.

A useful heuristic filter on the `OsiFeatures` to identify barrier-friendly instances is:

$$\text{OFdensity} > 1.0\%, \quad \text{OFrowNzAvg} > 50, \quad \text{OFrows} > 5000, \quad \text{OFpercEqualities} > 40\%.$$

9.3 Candidate MIPLIB 2017 Instances for Barrier Experiments

Table 2 lists MIPLIB 2017 instances whose LP relaxations have structural properties that suggest barrier may be competitive or superior to simplex.

Table 2: MIPLIB 2017 instances likely to benefit from barrier for the LP relaxation.

Category	Instance	Rationale
Dense / large-scale	neos-5075914-elvire	Large, dense constraint matrix
	neos-3083819-nubia	High density, many nz per row
	supportcase18	Large-scale, dense blocks
	graph40-80-1rand	Dense graph-based structure
Network / flow	mc11	Multicommodity flow
	momentum1	Large network structure
	neos-4954672-berkel	Flow-like, many equalities
	supportcase6	Large flow/network relaxation
Highly degenerate	neos-3627168-kasai	Degenerate LP, simplex stalls
	neos-4338804-snowy	Degenerate, simplex struggles
	seymour	Classic difficult degenerate LP

9.4 Proposed Training Pipeline for an Improved Decision Tree

The infrastructure for a learned replacement of the hand-crafted tree is largely in place. The following pipeline is proposed:

1. **Feature extraction.** Use `CbcInstanceFeatures::extract()` and `writeCsv()` on a large benchmark set (MIPLIB 2017, Mittelmann, Netlib) to obtain the 207 `OsiFeatures` per instance.
2. **Configuration space.** Define the candidate LP configurations, including at minimum the current four leaves plus barrier:
 - `-idiot 30 -pertvalue -1483 -primals`
 - `-idiot 60 -primals`
 - `-dualpivot pesteepest -psi 1.0 -pertv 52 -duals`
 - `-idiot 80 -primals`
 - `-barrier`
3. **Benchmark.** Solve each instance’s LP relaxation with every candidate configuration. Record wall-clock time.
4. **Labelling.** For each instance, assign the label of the fastest configuration.
5. **Tree training.** Train an optimal decision tree using one of the methods in Table 3.
6. **Code generation.** Convert the learned tree into a C++ `if/else` block and replace the body of `ClpSimplexOther::guess()`.

The key advantage of decision trees over other model families (random forests, gradient boosting, neural networks) is that they compile to pure `if/else` C++ with zero runtime dependencies—matching the self-contained nature of the CLP/CBC codebase.

Table 3: Decision tree training methods suitable for embedding in C++.

Method	Notes
CART (greedy)	Simple, fast to train; may be suboptimal for small depth budgets.
Optimal Classification Trees (OCT)	IP-based method from Vilas Boas et al. (2019); provably optimal for a given depth.
GOSDT	Near-optimal sparse trees; good balance of optimality and scalability.